

AutoForge: A Self-Evolving Multi-Agent System for Reusable Tool Creation

Liu Xinquan

HKU

Student ID: 3036659637

Cheung Yu Lung

HKU

Student ID: 3035477616

Lyu Linze

HKU

Student ID: 3036658619

Code Repository: <https://github.com/SeanLIUXQ/COMP7608A-Project-AutoForge.git>

1 Abstract

AutoForge is an LLM-powered agentic tool system that investigates whether language models can move beyond one-off code generation and accumulate reusable, verifiable tools over time. The system first uses Tool-RAG to retrieve existing tools from a registry. If no suitable tool is found, a multi-agent forge pipeline plans, writes, verifies, packages, and stores a new Python tool. The generated tool is then available for subsequent warm queries through the normal retrieval path. AutoForge combines a FastAPI backend, Streamlit frontend, tool registry, dynamic tool routes, MCP-compatible helpers, sandboxed execution, benchmark evaluation, and real LLM agent experiments. We evaluate the system with a 320-query benchmark, live backend strategy comparisons, retrieval-threshold sweeps, Docker sandbox evidence, and 50 real DeepSeek `deepseek-v4-pro` cold/warm agent trials. The backend `full` strategy achieved 100% success on the 320-query live benchmark with 40.63% tool reuse, while the real agent experiment achieved 50/50 cold forge success and 46/50 warm reuse. These results provide empirical evidence that reusable tool accumulation is feasible for deterministic automation tasks, while also showing that warm reuse remains sensitive to retrieval quality, payload extraction, schema alignment, and strict output formatting.

Keywords: LLM agents; multi-agent systems; tool use; retrieval-augmented generation; code generation; sandbox verification.

2 Introduction

Large language models can understand natural language and generate useful code, but many LLM applications still behave as one-shot problem solvers. When a user asks for a data transformation, string operation, or small automation script, a code-capable LLM can often produce a temporary solution. However, that code is usually discarded after the interaction. A semantically similar future query may trigger another full code-generation process, paying the latency and token cost again while also introducing new opportunities for hallucination or execution failure.

This creates a gap between **code generation** and **tool accumulation**. Tool-augmented agents are powerful when the correct tool already exists, but their toolset is usually manually defined. Code agents can create new procedures on demand, but they do not automatically turn successful code into a reusable, discoverable, and safe tool. AutoForge is designed around this gap.

The project asks the following research question:

Can an LLM agent automatically forge, verify, store, and later reuse deterministic tools, so that recurring tasks shift from expensive cold code generation to cheaper warm retrieval and execution?

Our core hypothesis is that a tool cache backed by retrieval can convert recurring tasks from repeated LLM generation into tool lookup plus sandboxed execution. This hypothesis is related to retrieval-augmented generation, which augments parametric language models with external non-parametric memory [1]; tool-using LLMs, where models learn when and how to invoke external APIs [3]; and agent frameworks such as ReAct, which interleave reasoning and acting [2]. AutoForge differs from these lines of work by emphasizing persistent tool creation: once a tool is successfully generated and verified, it becomes part of the system’s future action space.

Table 1: Coverage of the project-report requirements.

Required component	Where it is addressed in this report
Project idea description and motivation	Sections 2 and 4 explain the one-shot code generation problem, the tool accumulation motivation, and the cold-to-warm reuse loop.
Specific model implementation	Section 5 describes the Tool-RAG router, query strategies, Planner/Coder/Verifier/Packager pipeline, Docker sandbox, backend, MCP helper, and frontend.
Evaluation results of the system	Sections 6–7 report the benchmark protocol, live backend comparison, threshold sweep, 50-trial real LLM agent experiment, figures, and screenshots.
Key observations and conclusions	Sections 8, 9, and 11 summarize what worked, what did not work, what surprised us, limitations, and final conclusions.

3 Related Work

3.1 Retrieval-Augmented Generation and Tool Use

Retrieval-Augmented Generation (RAG) improves language models by retrieving external information before generation [1]. AutoForge adapts this idea from document retrieval to *tool retrieval*: the retrieved object is not a passage but an executable capability with schema, metadata, source code, and invocation traces. This design is also connected to Toolformer, which studies how language models can learn to call external tools through API-style interactions [3]. In AutoForge, the model does not merely select a pre-defined API; it can create a new tool when retrieval fails.

3.2 LLM Agents and Multi-Agent Collaboration

LLM-based agents often combine reasoning, planning, tool invocation, and environmental feedback. ReAct demonstrates that interleaving reasoning traces and task-specific actions can improve interpretability and performance [2]. MetaGPT shows that role specialization can structure multi-agent collaboration for software-style workflows [4]. AutoForge follows the same general principle: the Planner and Coder agents perform generative synthesis, while the Verifier and Packager agents impose deterministic control over acceptance, schema extraction, and registry insertion.

3.3 Feedback, Program Synthesis, and Persistent Skills

Feedback-driven refinement is important for agent reliability. Reflexion uses verbal feedback to help language agents improve across trials [5], while Self-Refine shows that LLM outputs can be iteratively improved through self-feedback [6]. In code generation, Codex and HumanEval highlight the importance of evaluating generated programs by functional correctness rather than surface similarity [7]. Voyager further demonstrates that agents can accumulate executable skills over time in an open-ended environment [8]. AutoForge adopts the same broad motivation of skill accumulation, but applies it to deterministic Python tool creation, sandbox verification, and warm reuse through retrieval.

4 Project Idea and Motivation

AutoForge treats generated tools as persistent system assets rather than temporary code snippets. The intended loop is:

1. A user submits a natural-language task.
2. The backend searches the existing tool registry using Tool-RAG.
3. If a confident match exists, AutoForge extracts structured arguments and runs the matched tool.
4. If no match exists, the multi-agent forge path creates a new tool.
5. The generated code is verified in a sandbox.
6. If verification passes, the tool is packaged with source code, schema, metadata, README, and example input.
7. Future similar tasks can retrieve and reuse the new tool.

This gives AutoForge a concrete form of long-term capability growth. Each successful cold forge can become future warm-path capacity. The system is not intended to be a general autonomous coding assistant. We deliberately focus on

deterministic, tool-like tasks where expected outputs can be evaluated and where generated functions can be sandboxed. This scope makes the project narrow enough to evaluate rigorously while still demonstrating an important agentic pattern: *LLM generation plus persistent tool memory*.

5 System Design and Implementation

5.1 Overall Architecture

AutoForge has four main layers: (i) a user and API layer, (ii) a Tool-RAG fast path, (iii) a multi-agent forge slow path, and (iv) a tool distribution layer. The system-level routing is shown in Figure 1.

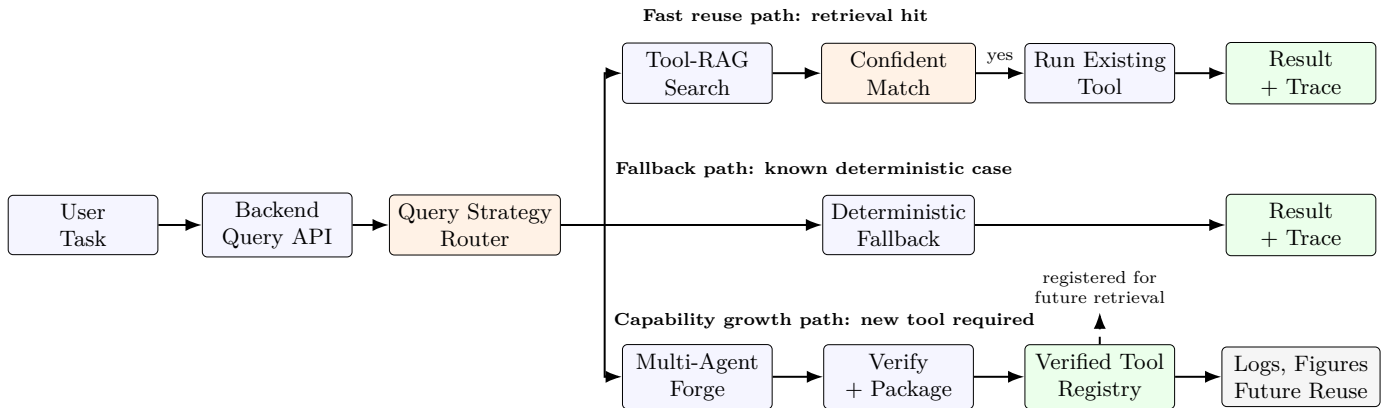


Figure 1: System-level routing in AutoForge. Retrieval serves the fast path, fallback preserves coverage for known deterministic cases, and successful forging expands the future registry.

5.2 Tool-RAG and Query Strategies

AutoForge uses Tool-RAG as a capability-retrieval layer rather than a document-answering layer. Each registered tool is stored as a skill bundle containing `metadata.json`, `schema.json`, `tool.py`, optional `README.md`, and example input. The registry converts this bundle into a searchable tool document that combines the tool name, natural-language description, parameter names, parameter descriptions, keywords, schema fields, and source-level capability hints. A user query is therefore compared against the operational interface of a tool, not against an arbitrary text passage.

The submitted live backend uses the default lexical retrieval path implemented in `backend/tool_registry.py`. Before scoring, quoted literals, list/dictionary payloads, numeric-only tokens, and common task words are down-weighted or removed so that retrieval focuses on user intent rather than benchmark payload values. The lexical score combines query-token overlap and a light length-ratio term, with the overlap component weighted more heavily. This design is deliberately simple and inspectable: when a query such as “reverse the text streamlit” is evaluated, the retrieval trace can show which tools matched because of intent words such as **reverse** and **text**, instead of being dominated by the literal string `streamlit`.

The project also includes an optional vector retrieval path. When vector store mode is enabled, `ToolRegistry` attempts to initialize ChromaDB and embeds tool documents using Sentence-Transformers. If ChromaDB or the embedding model is unavailable, the system falls back to lexical retrieval. In the final reported backend benchmark we kept the lexical path as the default because it is deterministic, lightweight, and easy for the TA to reproduce without first downloading embedding models.

Every retrieval returns a ranked candidate list and a `retrieval_trace` containing tool id, tool name, score, threshold, acceptance decision, and debug scores. The service only executes a candidate when its score reaches the configured similarity threshold. If the selected tool requires structured parameters rather than a single `query` argument, the backend uses the lightweight payload extractor in `backend/llm_tools.py` and validates the resulting payload against the tool schema before execution. This makes retrieval, argument extraction, and execution observable as separate sources of success or failure.

The backend supports four strategies: `full`, `no_retrieval`, `registry_only`, and `agent`. These strategies separate correctness, reuse, and coverage in the evaluation.

Table 2: Backend query strategies.

Strategy	Behavior	Purpose
full	Retrieve first; execute matched tool; fallback or forge on miss.	Main system behavior.
no_retrieval	Skip retrieval and use the slow/fallback path.	Baseline without reuse.
registry_only	Retrieve and execute only; no fallback.	Measures registry coverage.
agent	Force the LLM forge path.	Demonstrates cold tool creation.

This strategy design makes the evaluation more diagnostic. **full** measures the user-facing system with both reuse and fallback available. **no_retrieval** measures how much performance remains when reuse is disabled. **registry_only** intentionally removes fallback, so its failures reveal the current boundary of the tool catalog. **agent** forces cold tool creation and is used for the real DeepSeek-backed agent evidence. Together, these modes separate three research questions: whether AutoForge can answer the query, whether it can reuse an existing tool, and whether it can expand the registry when no suitable tool exists.

5.3 Multi-Agent Tool Forge

AutoForge uses a Planner-Coder-centered multi-agent framework for cold tool creation. In this report, an “agent” means a component with a specific responsibility, input/output contract, and state transition in the tool-forging workflow. Two agents are LLM-driven: the Planner converts the user request into an implementation plan and test intent, while the Coder synthesizes Python source code and a callable interface. The other agents are deterministic control agents: the Verifier runs static and sandboxed checks, and the Packager converts accepted code into a registry-compatible tool bundle. The Forge Orchestrator coordinates these agents, records trace evidence, and manages bounded repair attempts. The actual agent prompts used in the implementation are summarized in Appendix A.

This division is important because the LLM is used where open-ended synthesis is useful, but acceptance, execution, and packaging are controlled by reproducible code. If verification fails, the system does not accept the generated tool. Instead, it sends a typed error signal back to the Coder, allowing repair within a bounded retry loop. This makes the multi-agent path auditable: each stage has a concrete artifact, such as a plan, source file, verification result, sandbox log, schema, or packaged registry entry.

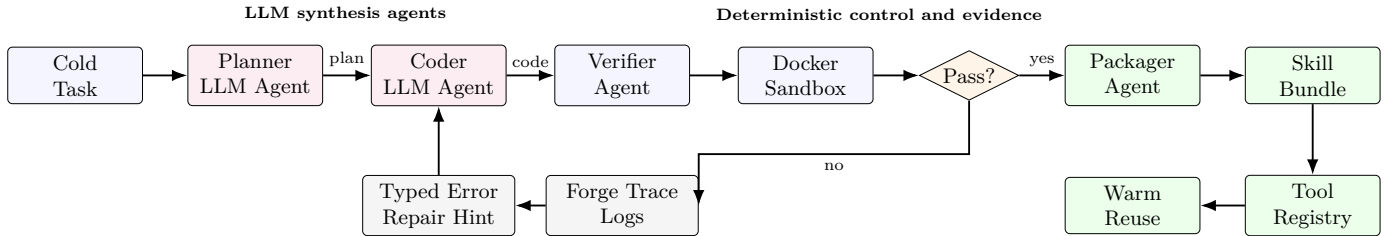


Figure 2: Internal multi-agent forge pipeline. Planner and Coder perform LLM synthesis; Verifier, Docker sandbox, Packager, and registry insertion provide deterministic control and evidence logging.

5.4 Specific Model Implementation

The implemented model path is a Planner-Coder-centered multi-agent framework backed by DeepSeek **deepseek-v4-pro** for the real LLM experiment. Here, “model” refers not only to the base LLM. It also includes the agent workflow, retrieval policy, verification contract, and tool artifact format used to turn natural-language tasks into reusable executable tools.

Table 3: Specific model implementation details.

Component	Concrete implementation
Base LLM for real-agent runs	DeepSeek <code>deepseek-v4-pro</code> , configured through <code>LLM_PROVIDER=deepseek</code> and <code>LLM_MODEL_NAME=deepseek-v4-pro</code> .
Agent framework	Planner-Coder-centered forge pipeline implemented in <code>agents/forge_pipeline.py</code> .
LLM-driven agents	Planner (<code>agents/planner.py</code>) produces implementation steps and function names; Coder (<code>agents/coder.py</code>) synthesizes reusable Python source code.
Deterministic control agents	Verifier (<code>agents/verifier.py</code>) runs static/sandbox checks and example tests; Packager (<code>agents/packager.py</code>) writes deployable skill bundles.
Retrieval policy	Tool-RAG registry search with threshold routing, exposed through backend strategies <code>full</code> , <code>registry_only</code> , <code>no_retrieval</code> , and <code>agent</code> .
Execution backend	Local sandbox for default verification; Docker sandbox for real-agent evidence and generated-code safety checks.
Prompt design	Planner, Coder, and Verifier prompts are stored under <code>agents/prompts/</code> and summarized in Appendix A.
Generated artifact	A reusable tool bundle containing <code>tool.py</code> , <code>schema.json</code> , <code>metadata.json</code> , <code>README.md</code> , <code>requirements.txt</code> , and <code>example_input.json</code> .

The input to the model path is a natural-language task plus optional structured payload. The output is not only an answer, but a packaged tool that can be registered and retrieved later. This implementation choice is central to AutoForge: the LLM performs open-ended synthesis during cold forge, while deterministic components decide whether the generated code is acceptable. As a result, the system can report both generation behavior and reuse behavior as separate measurable outcomes.

5.5 Sandbox, API, MCP, and Frontend

The backend is implemented as a Python FastAPI service with Pydantic request and response schemas shared through `shared/schemas.py`. `backend/main.py` assembles modular routers for health checks, tool listing/inspection, query execution, and dashboard evidence; the provided start scripts run the service with Uvicorn. The central orchestration logic lives in `backend/service.py`, while `backend/tool_registry.py` manages skill-bundle loading, retrieval, and registry synchronization. Generated tools are not treated as temporary snippets: `backend/dynamic_api.py` validates payloads against each tool’s JSON schema and can expose packaged tools as dynamic callable routes.

Sandbox execution is implemented in two layers. The default local runner performs AST parsing, rejects banned imports and calls, enforces timeouts, and checks JSON-serializable outputs. The Docker runner adds container-level isolation through the Docker CLI with `--network none`, `--read-only`, `--cpus 1.0`, `--memory 512m`, and `--pids-limit 256`. The real agent evidence uses Docker sandbox verification, while the default TA reproduction path can run locally without requiring Docker or an API key.

The MCP-compatible surface is implemented in `backend/mcp/server.py` using the `mcp` package’s `FastMCP` interface. It exposes three practical tool-facing operations: `list_forged_tools`, `invoke_forged_tool`, and `autoforge_query`. This lets an MCP client inspect the AutoForge catalog, invoke a registered generated tool directly, or route a natural-language query through the same backend service used by the REST API. Verification showed a non-empty catalog, successful direct invocation, and successful natural-language query routing.

The frontend is implemented with Streamlit and communicates with the backend through a small HTTP client in `frontend/api_client.py`. It contains a Chat page, Tool Browser, and Dashboard. The reporting and visualization pipeline uses Pandas for tabular summaries, Plotly for interactive dashboard-style charts, and Matplotlib for static report figures. Screenshots used as evidence are saved under `docs/report/real_agent_evidence/screenshots/`.

6 Benchmark and Evaluation Protocol

6.1 Dataset and Metrics

The main benchmark contains 80 canonical tasks and 240 paraphrases, giving 320 evaluation queries per strategy. The dataset is balanced across four difficulty levels and covers 17 tool families, including string operations, list operations, math, JSON processing, URL processing, CSV pipelines, JSONL pipelines, aggregation, table filtering, datetime operations, text

ETL, and unit conversion. The protocol separates a cold pass over canonical queries from a warm pass over paraphrases. This distinction is central because AutoForge evaluates not only whether a task is solved, but also whether the system can reuse accumulated capabilities. Representative dataset entries are provided in Appendix B.

Table 4: Benchmark composition.

Difficulty	Canonical	Paraphrases	Focus
L1	20	60	String operations, list operations, basic math, and unit conversion.
L2	20	60	JSON processing, URL parsing, timestamps, table filtering, and text grouping.
L3	20	60	CSV-like text, JSONL, aggregation, moving average, and multi-step processing.
L4	20	60	Longer inputs, complex filtering, aggregation, and multi-step ETL.

We report success rate, tool reuse rate (TRR), error rate, mean latency, speedup ratio, path distribution, per-family success, and categorized failure type. In the real agent evaluation, we additionally report cold forge success, warm task success, warm reuse, and cold/warm latency.

7 Evaluation Results

7.1 Live Backend Benchmark

The live backend benchmark was run on 320 queries per strategy. The main results are shown in Table 5.

Table 5: Live backend strategy comparison over 320 queries.

Strategy	Success	TRR	Error	Speedup	Path distribution
full	1.0000	0.4063	0.0000	1.1777	130 fast / 190 slow
no_retrieval	1.0000	0.0000	0.0000	0.0000	320 slow
registry_only	0.4063	0.4063	0.5938	0.0275	130 fast / 190 rejected or slow-like failures

The **full** strategy is the main system configuration. It achieved perfect correctness on this deterministic benchmark while reusing tools for 130 out of 320 queries. The **no_retrieval** baseline also achieved perfect correctness but had no reuse by design. The **registry_only** result shows the current registry coverage limit: when fallback is disabled, all retrieval misses become failures. Thus, retrieval improves efficiency when the registry already contains the relevant capability, whereas fallback or forging preserves coverage when the registry is incomplete.

This comparison is important because it separates two often-confused questions in agent evaluation. The first question is whether the system can produce correct outputs; the second is whether it can solve repeated tasks by reusing accumulated capabilities rather than regenerating solutions. Both **full** and **no_retrieval** can reach high correctness on this controlled benchmark, but only **full** demonstrates tool accumulation. Conversely, **registry_only** reveals that a reusable-tool system cannot be evaluated only by the quality of its individual tools: the breadth of registry coverage directly determines how often the fast path is available. This makes the registry itself an evolving model component, not merely a storage directory.

7.2 Threshold Sweep

The threshold sweep tested how retrieval aggressiveness affects path decisions. The results in Table 6 show a direct relationship between threshold selection and reuse.

Table 6: Retrieval threshold sweep.

Threshold	Success rate	Fast rate	Fast incorrect rate
0.55	1.0000	0.7000	0.0000
0.65	1.0000	0.6500	0.0000
0.75	1.0000	0.6250	0.0000
0.85	1.0000	0.0000	0.0000

Lower thresholds increase fast-path usage, while an overly strict threshold eliminates reuse. The *fast incorrect rate* is zero in all rows because, in this sweep, every query that was accepted by the fast path also matched the ground-truth output under the evaluation judge. Retrieval misses or low-confidence cases were routed to the slow/fallback path and therefore were not counted as fast incorrect. This does not prove that retrieval cannot make false-positive mistakes; it means that no such mistakes appeared in the 80 canonical sweep cases and seeded-tool registry used for Table 6. A production deployment would still need broader adversarial and out-of-distribution testing before relying on aggressive retrieval settings.

The sweep also gives a useful design insight: retrieval threshold is not a minor hyperparameter, but a control knob over the system’s behavior. A low threshold turns AutoForge into a more reuse-oriented system, while a high threshold turns it into a more conservative generation/fallback-oriented system. In practical deployments, this threshold could be adapted by domain: high-risk tasks should favor precision and verification, while low-risk repetitive transformations can favor higher reuse. This makes threshold calibration a meaningful part of the model implementation rather than a purely engineering detail.

7.3 Real LLM Agent Experiment

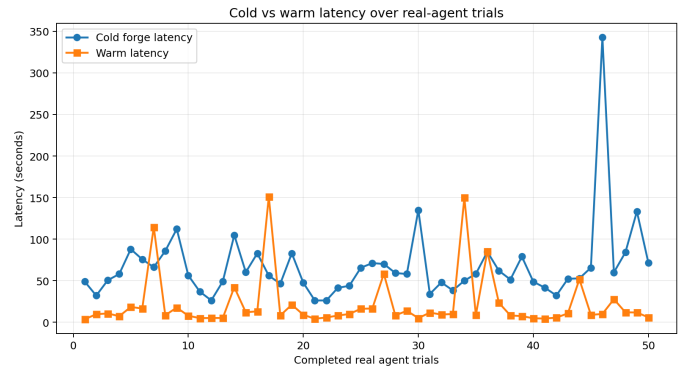
The final real agent experiment used DeepSeek **deepseek-v4-pro** with Docker sandboxing. It repeated 10 representative demo cases for five rounds, producing 50 real cold/warm trials. Table 7 summarizes the outcome.

Table 7: Summary of 50 real DeepSeek-backed agent trials.

Metric	Result
Total real trials	50
Base cases / repeats	10 / 5
Cold forge success	50/50
Cold forge success rate	1.0000
Warm task success	46/50
Warm task success rate	0.9200
Warm reuse	46/50
Warm reuse rate	0.9200
Duration	74.85 minutes
Mean cold latency	67.077 seconds
Mean warm latency	21.991 seconds



(a) Cumulative success and reuse rates.



(b) Cold and warm latency trend.

Figure 3: Real-agent trial dynamics generated from the raw 50-trial JSON report.

Figure 3a shows that cold forge success remained stable at 1.0 throughout the 50-trial run, while warm success and warm reuse converged to 0.92. This pattern indicates that the LLM-backed generation stage was not the dominant source of failure in this experiment. Instead, the remaining errors appeared after a tool had already been generated, during warm retrieval or argument extraction.

The research value of this result is that it decomposes “agent reliability” into at least two layers. The first layer is generative reliability: whether the LLM can synthesize a working tool when no tool exists. The second layer is reuse reliability: whether the system can later recognize that the tool is relevant and bind the new user input to the correct

schema. AutoForge performed strongly on the first layer and slightly less strongly on the second layer. This suggests that, for deterministic tool-building agents, the main bottleneck may shift from code generation to tool memory, retrieval, and interface alignment once a reasonable verifier is available.

Figure 3b illustrates the practical value of reuse. Mean cold latency was 67.077 seconds, compared with 21.991 seconds for warm execution, giving an approximate $3.05\times$ latency reduction for warm queries. The fifth repeat had noticeably higher cold latency, which is consistent with variability from real LLM API calls. Warm latency was lower and more stable because it avoided most of the planner and coder generation workload.

The latency result supports the central amortization argument of AutoForge. Cold generation is expensive because it includes planning, code synthesis, sandbox execution, repair attempts, and packaging. Warm execution is cheaper because these costs have already been paid. Therefore, the value of a forged tool increases with repeated use. This is a different efficiency story from ordinary prompt optimization: AutoForge reduces future work by changing the system state, not only by shortening the prompt.

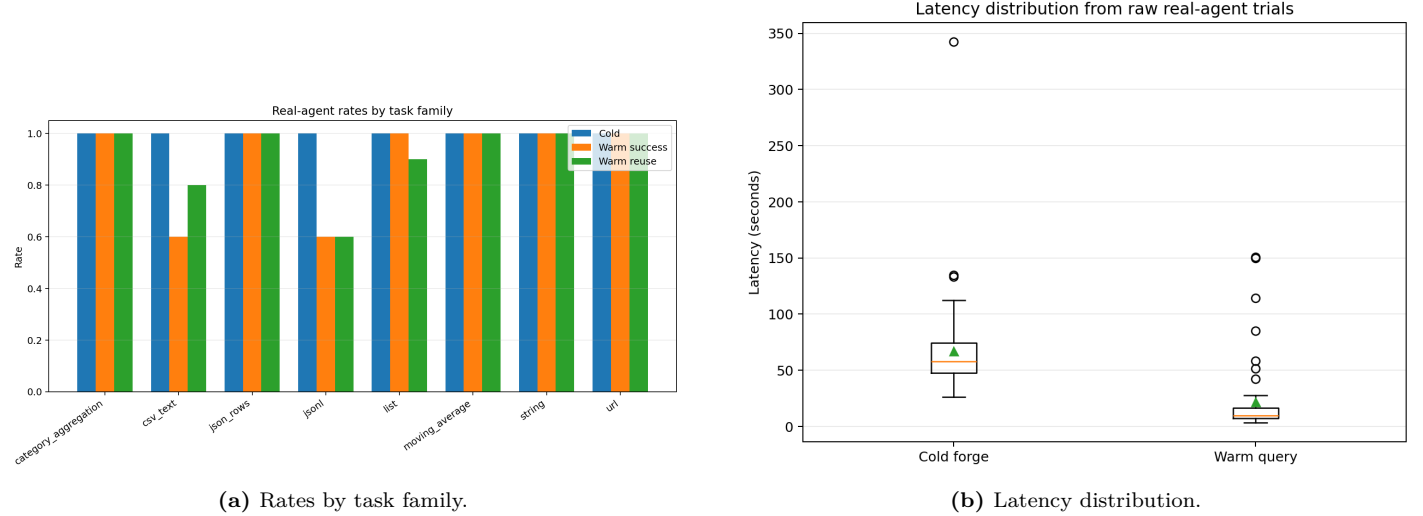


Figure 4: Per-family reliability and latency distribution in the real-agent experiment.

Figure 4a shows that most task families reached perfect warm success and reuse, including string, URL, JSON row, aggregation, and moving-average tasks. The weaker families were `csv_text` and `jsonl`, both of which reached only 0.6 warm success. This suggests that serialized, literal-heavy input formats are harder for the current retrieval and payload extraction pipeline than compact structured inputs. Figure 4b further separates cold and warm behavior: cold trials had a wider spread because they depended on LLM planning, code generation, verification, and packaging, while warm trials were concentrated at lower latency values.

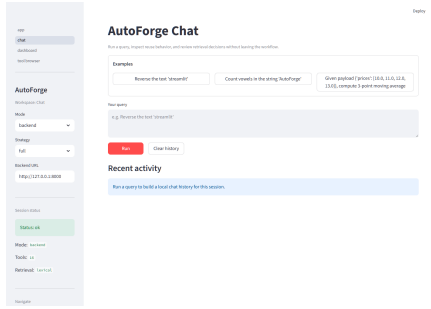
The family-level result is especially useful because it points to a concrete research direction. Failures were not evenly distributed across all tasks; they concentrated in formats where task intent and payload are interleaved in long strings. This implies that improving AutoForge is less about adding a larger model and more about improving the interface between language, schemas, and structured payloads. Better payload parsers, schema-aware retrieval, canonical output normalization, or format-specific adapters may produce larger gains than simply increasing the number of generated tools.

Taken together, these results show why self-evolving tool systems should be evaluated as stateful systems rather than as isolated prompt responses. A single success-rate number hides whether the system is solving tasks by regeneration, deterministic fallback, or accumulated memory. In AutoForge, the most research-relevant signal is the cold-to-warm transition: a successful cold forge changes the future action space by adding a verified tool to the registry. The remaining warm-side failures therefore become valuable diagnostic data, because they reveal the boundary between *tool creation* and *tool reuse*. This distinction is important for future agent research: once generation is reliable enough, the hard problem shifts toward memory indexing, schema grounding, and robust invocation under varied natural-language phrasing.

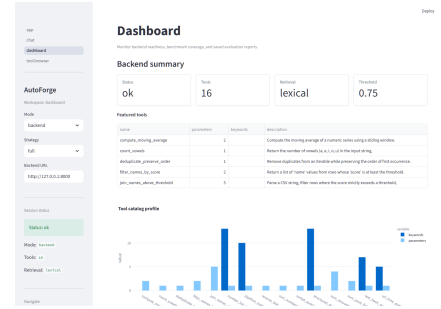
Table 8: Interpretation of the main evaluation signals.

Signal	Observed result	Research implication
Cold forge reliability	50/50 generated tools passed Docker-backed verification.	Multi-agent decomposition plus deterministic verification can make LLM code synthesis reliable for bounded tool-like tasks.
Warm reuse reliability	46/50 warm cases reused the generated tool successfully.	Persistent tool memory works, but reuse quality depends on retrieval, schema binding, and payload extraction.
Latency amortization	Warm execution was about $3.05\times$ faster than cold forging.	The system gains value over repeated use because expensive generation and verification costs are paid once and reused later.
Family-level failures	CSV-like text and JSONL were weaker than compact structured tasks.	Literal-heavy inputs expose a language-to-schema interface problem rather than a pure model-size problem.

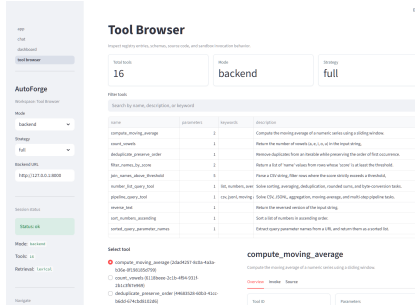
7.4 Frontend, API, and System Evidence



(a) Streamlit Chat page.



(b) Dashboard evidence page.



(c) Tool Browser.



(d) FastAPI OpenAPI page.

Figure 5: Executable system evidence: frontend interaction, dashboard reporting, generated-tool inspection, and backend API documentation.

These screenshots show that AutoForge is not only a script-based experiment. It includes a deployed backend, user-facing interface, dashboard, tool browser, and API documentation. This is important because the evaluation evidence is tied to an executable system rather than an isolated notebook or offline simulation.

8 Key Observations and Discussion

What worked. The cold forge pipeline worked reliably in the real LLM experiment. Across 50 DeepSeek-backed trials, every cold query produced a sandbox-verified packaged tool. This suggests that the Planner-Coder-Verifier-Packager separation is effective for deterministic tool-like tasks, especially when creative generation is separated from deterministic verification. Tool-RAG also worked as intended at the system level: the live backend `full` strategy reused tools for 40.63% of the 320 benchmark queries while preserving 100% success. The dashboard and trace design improved observability by separating generation failures from reuse failures. Docker sandbox evidence strengthened the safety argument by showing that safe generated code can run, while unsafe file access and unsafe imports are rejected.

What did not work. Warm reuse is harder than cold generation. Even when a generated tool passes verification, later reuse still depends on retrieval, schema matching, and payload extraction. In the 50-trial real agent experiment, four warm trials failed exact success or reuse. Literal-heavy formats such as CSV text and JSONL were the most fragile because task intent is mixed with raw serialized data. Exact string evaluation can also be strict: a semantically reasonable CSV output may fail exact matching because of whitespace or delimiter differences. Finally, registry-only execution is too strict for complete coverage, because disabling fallback converts all retrieval misses into failures.

What surprised us. The strongest surprise was that real cold forge performance was more stable than expected. We anticipated more LLM variability across 50 repeated generations, but the cold forge stage succeeded every time under Docker sandbox verification. The likely reason is that the task scope was deliberately constrained to deterministic Python functions with clear input/output contracts, and the Planner-Coder-Verifier separation gave the model structured guidance plus immediate executable feedback.

Another surprise was that the bottleneck moved from code generation to reuse orchestration. Once the agent can generate working functions, the next hard problem is making future natural-language requests invoke the right generated tool with the right structured payload. This happened because warm reuse depends on several non-generative components at once: retrieval scoring, schema alignment, payload extraction, and exact output normalization. A correct generated tool is therefore necessary but not sufficient for successful reuse.

Finally, threshold sensitivity was clearer than expected. A strict threshold of 0.85 removed fast-path reuse entirely in the sweep, while lower thresholds increased fast-path use without producing fast incorrect cases in this benchmark. The reason is that the benchmark tasks are semantically related but often phrased with different surface forms; a high threshold over-penalizes paraphrases, while a moderate threshold better captures reusable intent without accepting wrong tools in this controlled dataset.

9 Limitations

AutoForge is still a proof-of-concept system. The benchmark focuses on deterministic tool-like tasks, so the conclusions should not be generalized to open-ended reasoning, interactive web tasks, or tasks requiring external side effects. The 50 real agent trials do not replace a full 320-query `strategy=agent` benchmark. Because of time constraints and token-cost considerations, this experiment used 50 real agent tests and selected the latest domestic large language model, DeepSeek `deepseek-v4-pro`, as the base model. Future work will compare models from multiple LLM providers and evaluate which model family performs best within the AutoForge tool-forging workflow.

The current retrieval setup is mainly lexical in the live run. Optional vector retrieval exists, but the final reported backend run used lexical retrieval. Future work should compare lexical-only, vector-only, and hybrid retrieval more rigorously. Finally, the sandbox rejects obvious unsafe imports and calls, but it is not a formal security proof. Production deployment would require stronger isolation, stricter dependency control, audit logging, and container-level policy enforcement beyond this course prototype.

10 Individual Contributions

Liu Xinquan served as the project lead and took primary responsibility for the overall technical direction, final integration, and real-agent validation. His work covered the core multi-agent forge path, including the Planner/Coder/Verifier/Packager workflow, DeepSeek-backed agent execution, Docker sandbox evidence, 50-trial real LLM experiment design, result interpretation, and final report integration. He also coordinated the transition from a deterministic prototype to a real LLM-backed demonstration with reproducible evidence. This was the largest integration responsibility in the project because it connected model calls, generated code, sandbox execution, registry packaging, experimental logs, and the final written analysis.

Cheung Yu Lung focused on the backend and tool-serving infrastructure. His contribution included the FastAPI query path, Tool-RAG retrieval logic, query strategy design, dynamic tool route behavior, MCP-compatible helper surface, and threshold sweep analysis. He also helped define the behavior of `full`, `no_retrieval`, `registry_only`, and `agent` strategies, which became central to the evaluation protocol. This work made it possible to evaluate AutoForge as a deployable tool server rather than only as an offline code-generation script.

Lyu Linze focused on the frontend, benchmark, evaluation, and presentation layer. His contribution included the Streamlit Chat, Tool Browser, and Dashboard pages; benchmark dataset organization; evaluation metrics and reporting; visualization assets; and failure-analysis support. He helped make the 80 canonical tasks, 240 paraphrases, 17 tool families, strategy comparison tables, and dashboard screenshots usable as report evidence. This work made the system observable and made the final results easier to explain through tables, figures, screenshots, and dashboard evidence.

Table 9: Team member responsibilities.

Member	Role	Main Contribution
Liu Xinquan	Project lead; agent logic and algorithms	Overall coordination, multi-agent forge pipeline, Planner/Coder/Verifier/Packager workflow, Docker sandbox evidence, real DeepSeek experiment, 50-trial result analysis, and final report integration.
Cheung Yu Lung	Backend, RAG, and MCP	FastAPI backend, Tool-RAG retrieval, query strategies, dynamic routes, MCP-compatible helper, threshold sweep, backend routing analysis, and tool-serving behavior.
Lyu Linze	Frontend, benchmark, and evaluation	Streamlit frontend, dashboard, Tool Browser, 320-query benchmark organization, evaluation metrics, report assets, visualization, and failure explorer.

11 Conclusion

AutoForge shows that LLM agents can be organized as a stateful tool-building system rather than as a sequence of one-off prompt responses. The system combines Tool-RAG retrieval, a Planner-Coder-centered forge pipeline, deterministic verification, Docker sandbox evidence, tool packaging, FastAPI/MCP exposure, Streamlit visualization, and benchmark-driven evaluation. This integration is important because the generated artifact is not only an answer to the current query; after verification and packaging, it becomes a persistent capability that future queries can retrieve.

The evaluation supports the project hypothesis with several qualifications. On the 320-query backend benchmark, the full strategy preserved 100% task success while reusing existing tools for 40.63% of queries. In the 50-trial real DeepSeek experiment, every cold forge attempt produced a Docker-verified tool, and 46 of 50 warm queries successfully reused the generated capability. These results suggest that, for deterministic automation tasks, the main challenge is not only whether an LLM can write working code. Once sandboxed generation becomes reliable, the harder research problem shifts toward memory: deciding when a generated tool should be reused, binding natural-language payloads to the correct schema, and normalizing exact outputs across paraphrases.

The limitations are equally informative. AutoForge works best for bounded, deterministic tasks with clear expected outputs; it should not be interpreted as a general autonomous software engineer. Warm reuse was weaker on literal-heavy formats such as CSV-like text and JSONL, where task intent and payload content are entangled. The threshold sweep also shows that retrieval confidence is a behavioral control knob: lower thresholds increase reuse, while strict thresholds can eliminate the fast path entirely. Future work should therefore focus on schema-aware retrieval, stronger payload extraction, hybrid lexical/vector matching, richer adversarial benchmarks, and more formal sandbox hardening.

Overall, AutoForge provides a concrete prototype of a self-evolving tool server for LLM agents. Its central contribution is the cold-to-warm capability loop: create a tool when the registry is incomplete, verify it, store it, and reuse it later. This pattern turns tool generation from a transient model output into an accumulating system resource.

Reproducibility

The final full repository test run passed with 50 `passed`. Important result files include:

- `evaluation/results/eval_report_backend_full_20260501_060612.json`
- `evaluation/results/eval_report_backend_no_retrieval_20260501_060612.json`
- `evaluation/results/eval_report_backend_registry_only_20260501_060612.json`
- `evaluation/results/threshold_sweep_full_20260430_172632.json`
- `evaluation/results/agent_demo_report_agent_20260502_074909.json`
- `docs/report/real_agent_evidence/`

The real-agent figure generation command was:

```
python scripts\build_agent_demo_figures.py ^
```

```
--report evaluation\results\agent_demo_report_agent_20260502_074909.json ^
--output-dir docs\report\real_agent_evidence\figures
```

References

- [1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuettler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, 2020. <https://arxiv.org/abs/2005.11401>.
- [2] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*, 2023. <https://arxiv.org/abs/2210.03629>.
- [3] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, 2023. <https://arxiv.org/abs/2302.04761>.
- [4] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiaowu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Juergen Schmidhuber. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *International Conference on Learning Representations*, 2024. <https://arxiv.org/abs/2308.00352>.
- [5] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, 2023. <https://arxiv.org/abs/2303.11366>.
- [6] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, 2023. <https://arxiv.org/abs/2303.17651>.
- [7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating Large Language Models Trained on Code. arXiv preprint arXiv:2107.03374, 2021. <https://arxiv.org/abs/2107.03374>.
- [8] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv preprint arXiv:2305.16291, 2023. <https://arxiv.org/abs/2305.16291>.

Appendix A: Agent Prompts

The forge path is Planner-Coder-centered: the Planner and Coder are the two LLM-driven agents, while verification and packaging are deterministic control stages. The following prompts are taken from the project implementation under `agents/prompts/`.

A.1 Planner System Prompt

```
You are a planner for an autonomous tool-building system.
Return ONLY valid JSON with the exact keys:
- "steps": array of short strings
- "suggested_function_name": snake_case name

Rules:
```

- 1) Keep steps concise (4-7 items).
- 2) Function name should be a valid Python identifier in snake_case.
- 3) Prefer a reusable function name for the task family, not a name tied to one example value.
- 4) Include steps for input validation, deterministic processing, and returning a plain Python value.
- 5) Do not wrap the JSON in markdown or code fences.

Generalization few-shot task families:

- Text cleanup/transforms: reverse strings, count vowels, title-case phrases, keep only phone digits, top-k tokens.
- Number/list transforms: sort ascending, mean/average, deduplicate while preserving first occurrence, rounded sum, unit conversion.
- Structured JSON/dict tasks: sorted keys, contains-key checks, sum one field across rows, filter rows by score, group words by first letter.
- Pipeline tasks: apply percentage price changes, drop null values then aggregate, parse CSV text and join matching names.
- JSONL/list-of-lines tasks: parse each JSON line, keep valid records, sum a requested field.
- Time/URL tasks: sorted URL query keys and ISO timestamp interval in minutes.

When planning, name the reusable operation and expected argument shape. Do not encode literal values from the user request into the function name or plan.

A.2 Coder System Prompt

You are a Python code generator.

Given a plan, write a single self-contained Python function that generalizes well.

Requirements:

- 1) Use only the Python standard library.
- 2) Include type hints and a short docstring describing inputs/outputs.
- 3) Function name must match the plan's `suggested_function_name`.
- 4) Return results; do not print unless explicitly required.
- 5) Do not hardcode example inputs or constants tied to a single query.
- 6) Validate inputs where reasonable; fail fast with clear exceptions.
- 7) Avoid side effects (no file/network access).
- 8) Prefer JSON-serializable return values: `str`, `int`, `float`, `bool`, `list`, or `dict`.
- 9) Keep imports minimal; never import `os`, `sys`, `subprocess`, `socket`, `shutil`, or `pathlib`.
- 10) Avoid `open`, `eval`, `exec`, `compile`, `input`, and `__import__`.
- 11) Output exactly one Python code block using triple backticks.

Generalization patterns to imitate:

- For string tasks, accept the string as an argument and implement the family operation, not the exact quoted example.
- For list tasks, accept a list argument and handle any comparable or numeric values allowed by the operation.
- For row/JSON-like tasks, accept rows or dictionaries directly; validate keys before aggregation/filtering.
- For CSV text, parse with `csv.DictReader` and keep threshold/field names as parameters where useful.
- For JSONL, accept a list of JSON strings and parse each line with `json.loads`.
- For moving averages, accept values plus window size and validate that the window is positive and not longer than the series.
- For category aggregation, accept row dictionaries and configurable category/value field names with sensible defaults.

A.3 Verifier Prompt

You are a code verifier. Provide brief, actionable feedback on errors.

If the code is correct, respond with "OK".

Appendix B: Benchmark Dataset Examples

The benchmark dataset is stored in `evaluation/benchmark/dataset.json`. Each item contains a canonical query, three paraphrases, expected output, expected output type, difficulty level, and tool-family label. Table 10 gives representative examples across difficulty levels.

Table 10: Representative benchmark dataset examples.

ID	Family	Canonical query	Expected output
L1_001	string_ops	Count vowels in the string 'AutoForge'	5
L2_001	phone_cleanup	Normalize phone '+852 9123-4567' to digits only	85291234567
L3_001	text_stats	Find top 2 frequent tokens in 'a b c a b a'	[a, b]
L4_001	table_filter	Filter rows with names Kai, Li, Mo, and Jo where score ≥ 85 and return names	[Kai, Li, Jo]

For each canonical query, the dataset includes paraphrases. For example, L1_001 includes: “Return vowel count for 'AutoForge',” “How many vowels are in 'AutoForge'?,” and “Count vowels for 'AutoForge'.” These paraphrases support the cold/warm protocol by testing whether a capability learned from one phrasing can be reused for semantically equivalent requests.